

Borda Academy 2026

Project Report

Embedded Systems Developer Intern &
New Graduate Candidate Assignment

ŞULE NUR DEMİRDAŞ
12.04.2026

İçindekiler

1. Introduction	2
2. Hardware and Development Environment	2
2.1. Sensors and Documentation	2
2.2. Microcontroller and IDE	2
3. Software Architecture and Documentation.....	2
3.1. Flow Chart of the Architecture	4
4. Tasks and Solutions	5

1. Introduction

This project aims to collect environmental data from multiple I2C-based sensors, process this data using statistical methods on an STM32 microcontroller, and broadcast the results via Bluetooth Low Energy (BLE). The system is designed to be robust, memory-efficient, and maintainable.

2. Hardware and Development Environment

2.1. Sensors and Documentation

For this project, three different I2C sensors were selected to monitor the environment. Detailed technical specifications, register maps, and operating conditions for these sensors can be found in the **Datasheets/** folder within the submitted project ZIP file.

- **AHT20:** High-precision Temperature and Humidity sensor.
- **BH1750:** Ambient Light Intensity (Lux) sensor.
- **BMP180:** Barometric Pressure and Temperature sensor.

2.2. Microcontroller and IDE

- **MCU:** STM32F407G-DISC1 (ARM Cortex-M4).
- **IDE:** STM32CubeIDE was used for development, debugging, and static analysis.
- **Drivers:** The **STM32 HAL (Hardware Abstraction Layer)** library was utilized to manage I2C and UART peripherals, ensuring a standard and modular approach to hardware interaction.

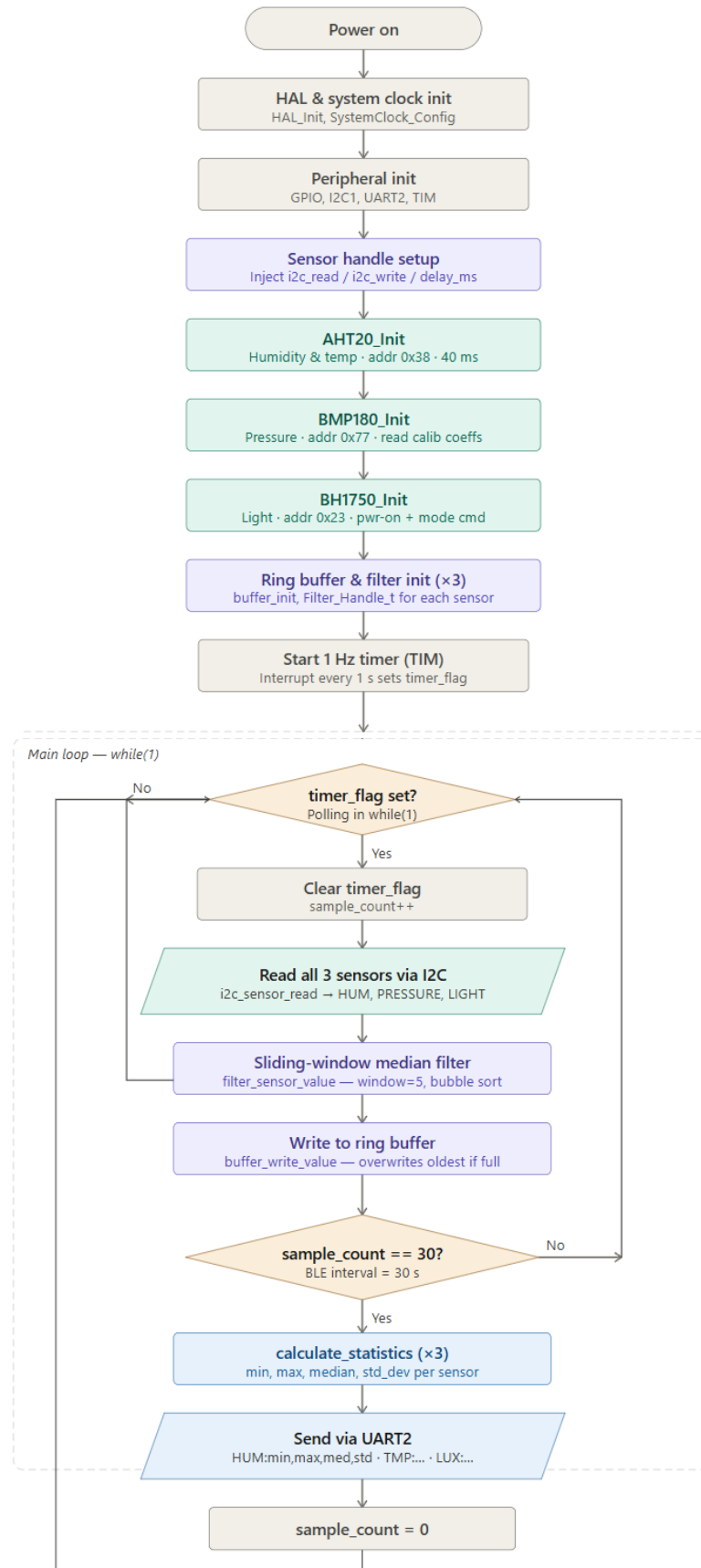
3. Software Architecture and Documentation

The software is designed with a modular approach, separating hardware-dependent drivers from high-level logic.

- **Doxygen Documentation:** For a deep dive into the code's logic, function descriptions, parameter details, and return values, [please refer to the Doxygen files provided in the Docs/ folder under the workspaces](#). Every core function (filtering, buffer management, statistics) is documented according to industry standards.

3.1. Flow Chart of the Architecture

MCU Firmware (C)



4. Tasks and Solutions

Task 1: Sensor Integration and Data Filtering

I2C communication was established using the STM32 HAL API to interact with the AHT20, BH1750, and BMP180 sensors. To mitigate high-frequency environmental noise and prevent outlier distortion, a moving average filter was implemented. Raw sensor readings are passed through the `filter_sensor_value` function, ensuring that only stable and reliable data is forwarded to the storage layer.

Task 2: Data Storage and Memory Management

A custom circular ring buffer (`buf_handle_t`) with a static size of 30 was developed. As new data arrives, the head pointer wraps around using modulo arithmetic ($\text{head} = (\text{head} + 1) \% \text{size}$), natively overwriting the oldest entries without requiring dynamic memory allocation.

NOTE: During continuous testing, a Stack Overflow leading to a HardFault was identified in the second 30-second cycle. This was successfully debugged and resolved by implementing a strict upper-bound boundary check ($\text{count} < \text{size}$) in the `buffer_write_value` function. Prevents count value to be larger than buffer size.

Task 3: Statistical Analysis

A dedicated `calculate_statistics` function was developed. It iterates through the valid elements in the ring buffer to extract the minimum, maximum, and mean values in a single pass. (Standard library “math” is used here) The variance is then computed to find the standard deviation using the `sqrtf()` function. To find the median without disrupting the chronological order of the ring buffer, a temporary local copy of the array is created on the stack and sorted using a custom **Bubble Sort** algorithm.

Task 4: BLE Broadcasting and Payload Formatting

Solution: To overcome the hardware constraint of the STM32F407 lacking an onboard BLE module, a UART-to-BLE Gateway architecture was implemented using a Raspberry Pi 4.

Due to the strict 31-byte limit of standard BLE advertisement packets, a custom compression method was utilized instead of using 32-bit floating-point numbers.

1. The STM32 sends the calculated statistics (Min, Max, Median, Std Dev) for 3 sensors (Humidity, Temperature, Light) via UART. (In the Figure 1 data received from Raspberry is shown.)

```

sule_pi@sule-raspberry:~ $ python3 sensor_bridge_ble.py
[UART] Bağlanıyor...
[UART] Bağlandı. Veri bekleniyor...
[UART] HUM:41.63,44.68,42.67,1.00
[UART] TMP:23.30,23.36,23.32,0.02
[UART] LUX:46.67,49.17,47.08,0.69
[BLE] Yayınlandı: HUM(42.67%) TMP(23.32°C) LUX(47.08lx)
[UART] HUM:43.12,45.20,44.48,0.63
[BLE] Yayınlandı: HUM(44.48%) TMP(23.32°C) LUX(47.08lx)
[UART] TMP:23.36,23.41,23.39,0.01
[BLE] Yayınlandı: HUM(44.48%) TMP(23.39°C) LUX(47.08lx)
[UART] LUX:47.50,49.17,49.17,0.30
[BLE] Yayınlandı: HUM(44.48%) TMP(23.39°C) LUX(49.17lx)
[UART] HUM:42.33,43.18,42.56,0.25
[BLE] Yayınlandı: HUM(42.56%) TMP(23.39°C) LUX(49.17lx)

```

Figure 1. Raspberry pi terminal screenshot.

2. The Raspberry Pi bridge script parses these values, multiplies each by 100, and casts them to **16-bit signed integers (int16)** to retain two decimal places of precision while saving space.
3. These values are packed in **Little-Endian** format. Each sensor requires 8 bytes (4 statistics x 2 bytes), totaling 24 bytes of sensor data.

Advertisement Packet Decoding

The resulting 24-byte payload is embedded into the Manufacturer Specific Data field. The complete payload structure observed in the scanning application is as follows:

- **Company ID:** 0xFFFF (Test/No Company ID)
- **Byte [0-7] (Humidity):** Min, Max, Median, Std Dev.
- **Byte [8-15] (Temperature):** Min, Max, Median, Std Dev.
- **Byte [16-23] (Light/Lux):** Min, Max, Median, Std Dev.

Validation via Mobile Scanner

To verify the broadcasting mechanism, the BLE packets were captured using a mobile BLE scanner application (nRF Connect).

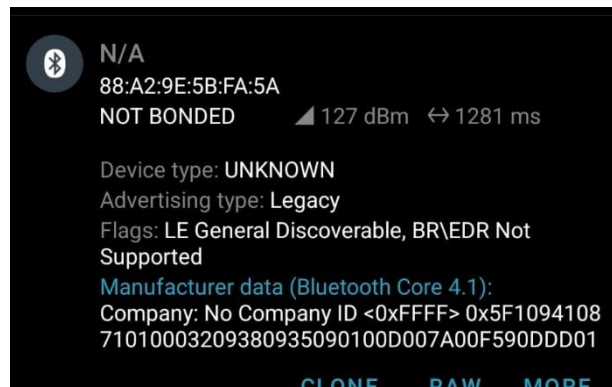


Figure 2: Observed data 1 on nRF Connect

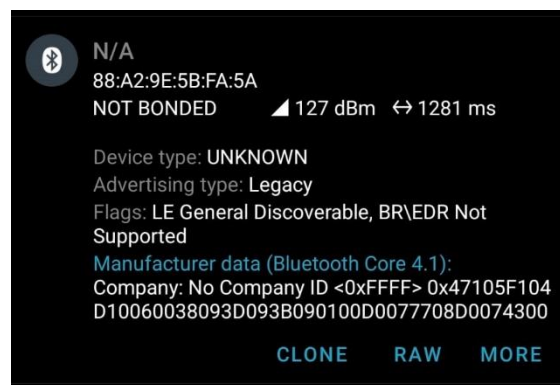


Figure 3: Observed data 2 on nRF Connect

Figure 1: BLE Scanner capturing the custom Manufacturer Specific Data broadcast by the Raspberry Pi Gateway.

In the first captured packet Figure 2, the Manufacturer Specific Data is observed as:

0xFFFF 0x4710 5F10 4D10 0600 3809 3D09 3B09 0100 D007 7708 D007 4300

Decoding this Little-Endian hex stream:

- **Humidity:** 0x1047 (4167 -> **41.67%**), 0x105F (4191 -> **41.91%**), 0x104D (4173 -> **41.73%**), 0x0006 (6 -> **0.06**)
- **Temperature:** 0x0938 (2360 -> **23.60°C**), 0x093D (2365 -> **23.65°C**), 0x093B (2363 -> **23.63°C**), 0x0001 (1 -> **0.01**)
- **Light:** 0x07D0 (2000 -> **20.00 Lux**), 0x0877 (2167 -> **21.67 Lux**), 0x07D0 (2000 -> **20.00 Lux**), 0x0043 (67 -> **0.67**)

The successful reception and decoding of these dynamic packets confirm that the STM32 processing, UART synchronization, and BLE broadcasting are functioning strictly according to the assessment requirements.

